SmartStock AI

Services & API Reference — Local Development Environment

Version 1.0.0 · Generated 2026-07-04 · Branch: release/1.0.0

> **What this document is:** a practical reference for accessing the SmartStock platform and its microservice APIs — service catalog, ports, authentication, endpoint reference, and copy-paste workflow examples. Intended as the go-to guide for future development and testing sessions.

1. Platform Overview

SmartStock is an inventory-management platform built as Spring Boot 3 / Java 21 microservices. All external traffic enters through the **API Gateway** (Spring Cloud Gateway) on `http://localhost:8080`, which also serves a built-in **web console UI** at its root. Each domain service owns its own PostgreSQL database (no shared databases), and services communicate through REST via the gateway plus asynchronous Kafka events.

Starting and stopping the stack

```
# from /home/youssef/Projects/SmartStock
docker compose up -d          # start everything (~30 containers)
docker compose ps             # health check — all app services report (healthy)
docker compose down           # stop (data volumes are preserved)
```

Note: a local `docker-compose.override.yml` caps JVM/Kafka memory for the 8 GB host — keep it when running locally.

2. Access Points

What	URL	Credentials
Web Console (main UI)	`http://localhost:8080`	`system.admin / Admin@SmartStock2026!`
REST API (gateway)	`http://localhost:8080/api/v1/…`	JWT Bearer token (see §3)
Grafana — metrics, logs, traces	`http://localhost:3000`	`admin / admin123`
Prometheus — raw metrics	`http://localhost:9090`	—
pgAdmin — database browser	`http://localhost:5050`	see `docker-compose.yml`
RabbitMQ management	`http://localhost:15672`	`guest / guest`
Mailpit — outgoing email capture	`http://localhost:8025`	—
MinIO console — object storage	`http://localhost:9001`	`minioadmin / minioadmin`

> **Security note:** these are development-only default credentials. Rotate the admin password via `POST /api/v1/users/{id}/change-password` before any shared or production deployment.

3. Authentication

Authentication is handled by the **Identity Service** (JWT + RBAC, per ADR-0005). Log in once, then send the access token on every request. Access tokens live **1 hour**; refresh tokens ~30 days.

```
# 1. Log in — returns data.accessToken and data.refreshToken
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"system.admin","password":"Admin@SmartStock2026!"}'

# 2. Use the token on every call
export TOKEN="<data.accessToken>"
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/products

# 3. Refresh when expired
curl -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" -d '{"refreshToken":"<data.refreshToken>"}'
```

Roles (seeded)

Role	Scope
`SYSTEM_ADMIN`	Full system access (the seeded `system.admin` user)
`WAREHOUSE_MANAGER`	Warehouse operations and staff management
`INVENTORY_OPERATOR`	Stock operations (stock-in/out, transfers, counts)
`SUPPLIER_MANAGER`	Supplier and purchase-order management
`REPORTER`	Read-only analytics and reporting
`AUDITOR`	Audit log access only

Response envelope

All services answer with a consistent envelope — check `data` on success, `errors[]` on failure:

```
{ "data": { … } | [ … ],  "meta": { "timestamp", "page", "size", "total", … },
  "errors": [ { "message", "code" } ]   // only on failure (4xx/5xx)
}
```

List endpoints paginate with `?page=0&size=20`. Every response carries a `correlationId` for tracing in Grafana/Tempo.

4. Service Catalog

Service	Port	Gateway route	Purpose	Status
API Gateway	8080	`/ (console), /api/v1/**`	Routing, console UI, single entry point	LIVE
Identity	8001	`/api/v1/auth, /users, /roles, /permissions`	Login, JWT, users, RBAC	LIVE

Product	8002	/api/v1/products	Catalog, SKUs, barcodes/QR, categories, import/export	LIVE
Inventory	8003	/api/v1/inventory	Stock levels, movements, adjustments, counts, reservations	LIVE
Warehouse	8004	/api/v1/warehouses	Warehouses, zones, shelves, bins, capacity	LIVE
Supplier	8005	/api/v1/suppliers	Suppliers, contacts, contracts, deliveries, performance	LIVE
Customer	8006	/api/v1/customers	Customers, contacts, addresses, segments	LIVE
Purchase Order	8007	/api/v1/purchase-orders	POs: create, confirm, receive, quality issues	LIVE
Sales Order	8008	/api/v1/sales-orders	SOs: create, confirm, pick, ship, deliver	LIVE
Notification	8009	/api/v1/notifications	Email/alert dispatch (via Mailpit locally)	PLANNED
Reporting	8010	/api/v1/reports	Business reports	PLANNED
Data Export	8011	/api/v1/exports	Bulk data exports	PLANNED
Analytics	8012	/api/v1/analytics	KPIs, demand forecasting	PLANNED
Audit	–	/api/v1/audit	Audit trail queries	PLANNED

"Planned" services exist as skeleton modules in `services/` with gateway routes pre-configured, but are not deployed in docker-compose yet. Note: the gateway route for `/api/v1/audit` currently points at port 8008 (sales-order's port) — fix before enabling the audit service.

Infrastructure (for direct access / debugging)

Component	Host port(s)	Notes
PostgreSQL (one per service)	5432–5439	identity 5432, product 5433, inventory 5434, warehouse 5435, supplier 5436, customer 5437, purchase-order 5438, sales-order 5439 — user/db `smartstock`
Kafka / Zookeeper	9092, 29092 / 2181	Domain events (user, product, stock movements…)
Redis	6379	Caching / rate limiting
RabbitMQ	5672 (AMQP), 15672 (UI)	Messaging
MinIO	9000 (S3 API), 9001 (UI)	Object storage (product images, exports)
Mailpit	1025 (SMTP), 8025 (UI)	Catches all outgoing email locally
Loki / Tempo	3100 / 3200, 4317–4318 (OTLP)	Logs and traces backing Grafana

5. API Endpoint Reference

All paths below are relative to `http://localhost:8080` (the gateway) and require the `Authorization: Bearer` header except login/refresh.

Identity — auth & user management

Method	Path	Description
POST	`/api/v1/auth/login`	Log in (username + password) → tokens
POST	`/api/v1/auth/refresh`	Exchange refresh token for a new access token
POST	`/api/v1/auth/logout`	Revoke the current refresh token
POST	`/api/v1/auth/forgot-password` · `/reset-password`	Password reset flow (email lands in Mailpit)
POST	`/api/v1/users/register`	Create a user account
GET	`/api/v1/users` · `/users/{id}`	List / read users
POST	`/api/v1/users/{id}/change-password` · `/deactivate` · `/reactivate`	Account lifecycle
POST	`/api/v1/users/{id}/roles`	Assign a role (DEL …/roles/{roleId} to revoke)
GET	`/api/v1/roles` · `/api/v1/permissions`	List roles / permissions

Product — catalog & barcodes

Method	Path	Description
POST	`/api/v1/products`	Create product (name, sku, price, cost, unit, reorder point…)
GET	`/api/v1/products`	List / search products (paged)
GET	`/api/v1/products/{id}`	Read by id
GET	`/api/v1/products/sku/{sku}`	Lookup by SKU
GET	`/api/v1/products/by-barcode/{barcode}`	**Scan:** lookup by barcode digits
PUT	`/api/v1/products/{id}`	Update product
POST	`/api/v1/products/{id}/barcode` · `/qrcode`	Generate barcode / QR code for the product
POST	`/api/v1/products/{id}/deactivate` · `/reactivate`	Lifecycle
POST	`/api/v1/products/import`	Bulk import (multipart CSV)
GET	`/api/v1/products/export`	Export catalog
GET POST	`/api/v1/products/categories` · `/categories/{id}`	Category management

Inventory — stock operations

Method	Path	Description
POST	/api/v1/inventory/stock-in	Receive stock (productId, warehouseId, quantity, unitCost)
POST	/api/v1/inventory/stock-out	Issue / ship stock out
POST	/api/v1/inventory/adjustments	Correct stock (± quantity, reason e.g. DAMAGED, LOST)
POST	/api/v1/inventory/transfers	Move stock between warehouses
POST	/api/v1/inventory/reservations	Reserve stock (e.g. for a sales order)
GET	/api/v1/inventory/stock/{productId}/{warehouseId}	Stock level for a product in one warehouse
GET	/api/v1/inventory/products/{productId}/stock	Stock for a product across all warehouses
GET	/api/v1/inventory/warehouses/{warehouseId}/stock	All stock in a warehouse
GET	/api/v1/inventory/transactions	Movement history (filterable)
POST	/api/v1/inventory/counts · /counts/{id}/items · /counts/{id}/complete	Physical inventory counting workflow

Warehouse — physical layout

Method	Path	Description
POST GET	/api/v1/warehouses · /{id}	Create / list / read warehouses (code, name, location…)
PUT	/api/v1/warehouses/{id}	Update; POST …/deactivate · /reactivate for lifecycle
GET	/api/v1/warehouses/{id}/capacity-report	Utilization report
POST GET	…/{id}/zones · …/zones/{z}/shelves · …/shelves/{s}/bins	Zones → shelves → bins hierarchy
GET	/api/v1/warehouses/{id}/bins/available	Find free bins

Supplier & Customer

Method	Path	Description
POST GET PUT DEL	/api/v1/suppliers · /{id}	Supplier CRUD; /suspend · /resume · /by-rating · /{id}/performance
POST GET	/api/v1/suppliers/{id}/contacts · /contracts · /deliveries	Related records; POST …/deliveries/{id}/confirm
POST GET PUT DEL	/api/v1/customers · /{id}	Customer CRUD; /suspend · /resume · /by-segment
POST GET	/api/v1/customers/{id}/contacts · /addresses · /orders	Related records

Purchase Orders & Sales Orders

Method	Path	Description
POST GET	`/api/v1/purchase-orders · /{id}`	Create / list / read POs
POST	`/api/v1/purchase-orders/{id}/confirm · /delivery · /cancel · /quality-issue`	PO lifecycle: confirm → receive delivery (feeds stock-in) → close
POST GET	`/api/v1/sales-orders · /{id}`	Create / list / read SOs
POST	`/api/v1/sales-orders/{id}/confirm · /pick · /shipments · /delivery/{shipmentId} · /cancel`	SO lifecycle: confirm → pick → ship → deliver
GET	`/api/v1/sales-orders/by-customer/{customerId} · /pending-delivery`	Queries

6. Common Workflows (copy-paste)

A. Scan a product and check its stock

```
# barcode scanner produces the digits — look the product up
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/products/by-barcode/1807228305172
# → data.id is the productId

# stock across all warehouses
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/inventory/products/<productId>/stock
```

B. Create a product with a barcode

```
curl -X POST http://localhost:8080/api/v1/products \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"Wireless Mouse","sku":"SKU-00123","description":"Demo",
       "brand":"Acme","unitPrice":29.99,"unitCost":12.50,"unit":"PIECE",
       "reorderPoint":10,"reorderQuantity":50}'
# then generate its barcode image:
curl -X POST -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/products/<productId>/barcode
```

C. Receive stock (stock-in)

```
curl -X POST http://localhost:8080/api/v1/inventory/stock-in \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"productId":"<productId>","warehouseId":"<warehouseId>",
       "quantity":100,"unitCost":12.50,"referenceType":"MANUAL","notes":"Initial stock"}'
```

D. Adjust stock (damage, loss, correction)

```
curl -X POST http://localhost:8080/api/v1/inventory/adjustments \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"productId":"<productId>","warehouseId":"<warehouseId>",
       "adjustmentQuantity":-5,"reason":"DAMAGED","notes":"Broken in storage"}'
```

E. Ship stock out / transfer between warehouses

```
curl -X POST http://localhost:8080/api/v1/inventory/stock-out \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"productId":"<productId>","warehouseId":"<warehouseId>","quantity":10}'

curl -X POST http://localhost:8080/api/v1/inventory/transfers \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"productId":"<productId>","sourceWarehouseId":"<from>",
       "targetWarehouseId":"<to>","quantity":20}'
```

F. Audit trail — what happened to my stock?

```
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8080/api/v1/inventory/transactions?productId=<productId>&page=0&size=20"
```

7. Current Local Data & Known Notes

Item	Value
Demo products	Wireless Mouse `SKU-FCULR` (barcode `1807228305172`), `SKU-DEMO2` (barcode `4299857208899`)
Demo warehouse	`WH-MAIN` — “Main Warehouse”, id `349fe968-d0bc-4ff5-9626-587ac3b2b8f5`
Seeded users	`system.admin` (SYSTEM_ADMIN), `preview.user`, `demo.admin`

> **Known issues to fix:** (1) The BCrypt hash in `V2__seed_data.sql` does not match the documented admin password — it was corrected directly in the local database on 2026-07-04, but the migration file still needs regenerating for fresh installs. (2) The gateway’s audit route points to port 8008, which belongs to sales-order. (3) See `KNOWN_LIMITATIONS.md` for the full list.

Sources: `docker-compose.yml`, `services/*/src/main/java/**/*Controller.java`, `services/api-gateway/src/main/resources/application.yml`, `services/identity-service/src/main/resources/db/migration/V2__seed_data.sql`. All endpoints in §5–6 were verified live against the running stack on 2026-07-04.